

# Spring Interceptors — Complete Notes

---

## 1. What is an Interceptor?

---

An **Interceptor** in Spring is a component that allows you to intercept HTTP requests **before** they reach a controller, **after** the controller processes them but **before** the view is rendered, and **after** the complete request has finished (view rendered/response sent).

Interceptors work at the **Spring MVC (DispatcherServlet) level**, not at the raw servlet container level. This gives them access to the Spring context (handler method, model, view, beans, etc.), unlike Servlet Filters which operate purely at the HTTP request/response level.

**Package:** `org.springframework.web.servlet.HandlerInterceptor`

---

## 2. Why Use Interceptors?

---

Interceptors let you implement **cross-cutting concerns** without polluting controller logic:

- Authentication & authorization checks
- Logging and auditing
- Performance/latency monitoring
- Locale or theme resolution
- Header/response manipulation
- Request validation
- Rate limiting

This keeps controllers clean and focused only on business logic (Separation of Concerns).

---

## 3. The `HandlerInterceptor` Interface

---

```
public interface HandlerInterceptor {  
  
    default boolean preHandle(HttpServletRequest request,  
                             HttpServletResponse response,  
                             Object handler) throws Exception {  
        return true;  
    }  
  
    default void postHandle(HttpServletRequest request,
```

```

        HttpServletResponse response,
        Object handler,
        @Nullable ModelAndView modelAndView) throws Exception {
    }

    default void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response,
                                Object handler,
                                @Nullable Exception ex) throws Exception {
    }
}

```

### 3.1 preHandle()

- Called **before** the controller (handler) method executes.
- Return `true` → request processing continues.
- Return `false` → request processing **stops immediately**; the interceptor itself is responsible for sending a response (e.g., error, redirect). Controller and remaining interceptors' `preHandle` (later in chain) are never invoked.
- Typical use: authentication, authorization, request validation, logging start time.

### 3.2 postHandle()

- Called **after** the controller method executes, but **before** the view is rendered.
- Gives access to the `ModelAndView` — you can add attributes to the model or modify the view name here.
- **Not called** if `preHandle()` returned `false`, or if an exception occurred in the handler.
- Typical use: adding common model attributes (e.g., logged-in user info to every view), modifying response headers before rendering.

### 3.3 afterCompletion()

- Called **after** the complete request has finished — i.e., after the view has been rendered.
- Called regardless of whether the handler threw an exception (the exception is passed as a parameter, `null` if none).
- Only called if that interceptor's `preHandle()` returned `true`.
- Typical use: cleanup (releasing resources, closing DB connections/ThreadLocals), final logging, performance metric recording (end time - start time).

## 4. Execution Order (Interceptor Chain)

Given interceptors **A**, **B**, **C** registered in that order:

Request →

A.preHandle() → B.preHandle() → C.preHandle()

→ Controller Method Executes

C.postHandle() → B.postHandle() → A.postHandle()

→ View Rendered

C.afterCompletion() → B.afterCompletion() → A.afterCompletion()

→ Response

### Key rules:

- preHandle() executes in **registration order** (A → B → C).
- postHandle() and afterCompletion() execute in **reverse order** (C → B → A) — similar to a stack/nested structure.
- If B.preHandle() returns false:
  - C.preHandle() is never called.
  - The controller is never invoked.
  - afterCompletion() is called only for interceptors whose preHandle() already returned true (i.e., only A.afterCompletion()), in reverse order.
  - postHandle() is **not called for any interceptor** in this case.

## 5. Implementing an Interceptor — Step by Step

### Step 1: Create the Interceptor class

```
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.lang.NonNull;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class LoggingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(@NonNull HttpServletRequest request,
                           @NonNull HttpServletResponse response,
                           @NonNull Object handler) {
        long startTime = System.currentTimeMillis();
        request.setAttribute("startTime", startTime);
        System.out.println("Request URL: " + request.getRequestURI());
        return true; // continue processing
    }
}
```

```

@Override
public void postHandle(@NonNull HttpServletRequest request,
                      @NonNull HttpServletResponse response,
                      @NonNull Object handler,
                      org.springframework.web.servlet.ModelAndView modelAndView) {
    System.out.println("Controller executed, view about to render");
}

@Override
public void afterCompletion(@NonNull HttpServletRequest request,
                          @NonNull HttpServletResponse response,
                          @NonNull Object handler,
                          Exception ex) {
    long startTime = (Long) request.getAttribute("startTime");
    long endTime = System.currentTimeMillis();
    System.out.println("Time taken: " + (endTime - startTime) + " ms");
    if (ex != null) {
        System.out.println("Exception occurred: " + ex.getMessage());
    }
}
}

```

## Step 2: Register the Interceptor

Implement `WebMvcConfigurer` and override `addInterceptors()`:

```

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final LoggingInterceptor loggingInterceptor;
    private final AuthInterceptor authInterceptor;

    public WebConfig(LoggingInterceptor loggingInterceptor, AuthInterceptor authInterceptor) {
        this.loggingInterceptor = loggingInterceptor;
        this.authInterceptor = authInterceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(loggingInterceptor)
            .addPathPatterns("/**") // apply to all URLs
            .excludePathPatterns("/login", "/error"); // skip these

        registry.addInterceptor(authInterceptor)
            .addPathPatterns("/api/**")
    }
}

```

```
        .order(1); // controls order relative to other interceptors
    }
}
```

## Key registration methods:

| Method                                      | Purpose                                              |
|---------------------------------------------|------------------------------------------------------|
| <code>addPathPatterns(String...)</code>     | Restrict interceptor to specific URL patterns        |
| <code>excludePathPatterns(String...)</code> | Exclude specific URLs from interception              |
| <code>order(int)</code>                     | Explicitly set execution priority (lower runs first) |

## 6. Practical Use Case Examples

### 6.1 Authentication/Authorization Interceptor

```
@Component
public class AuthInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler
        throws Exception {
        String token = request.getHeader("Authorization");
        if (token == null || !isValidToken(token)) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("Unauthorized: Invalid or missing token");
            return false; // stop the chain
        }
        return true;
    }

    private boolean isValidToken(String token) {
        // token validation logic (JWT parsing, DB check, etc.)
        return token.startsWith("Bearer ");
    }
}
```

### 6.2 Performance Monitoring

Record start time in `preHandle`, calculate duration in `afterCompletion` (shown above). Useful for logging slow endpoints or feeding metrics to Micrometer/Prometheus.

## 6.3 Adding Common Model Attributes

```
@Override
public void postHandle(HttpServletRequest request, HttpServletResponse response,
                        Object handler, ModelAndView modelAndView) {
    if (modelAndView != null) {
        modelAndView.addObject("appVersion", "1.0.0");
        modelAndView.addObject("currentYear", java.time.Year.now().getValue());
    }
}
```

## 6.4 Locale/Theme Resolution

Spring itself uses interceptors internally, e.g., `LocaleChangeInterceptor`, to switch locale based on a request parameter:

```
@Override
public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(new LocaleChangeInterceptor());
}
```

## 6.5 Audit Logging

Log which user accessed which endpoint, with timestamp, IP address, and request parameters — useful for compliance/security audits.

## 6.6 CORS Handling (custom, if not using `CorsFilter`)

Add necessary CORS headers in `preHandle` before the request reaches the controller.

## 6.7 Rate Limiting

Track request counts per user/IP in `preHandle`, and reject (return `false`) with HTTP 429 if the limit is exceeded.

---

# 7. Interceptors vs Filters vs AOP

---

| Aspect                              | Filter (Servlet)                                                     | Interceptor (Spring MVC)                               | AOP (Aspect)                                                  |
|-------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------|
| Layer                               | Servlet container (before DispatcherServlet)                         | Spring MVC (after DispatcherServlet routes to handler) | Method level (any Spring bean)                                |
| Spring context access               | Limited/None                                                         | Full access                                            | Full access                                                   |
| Access to handler method            | No                                                                   | Yes ( <code>HandlerMethod</code> object)               | Yes ( <code>JoinPoint</code> )                                |
| Access to <code>ModelAndView</code> | No                                                                   | Yes                                                    | No                                                            |
| Configuration                       | <code>web.xml</code> / <code>FilterRegistrationBean</code>           | <code>WebMvcConfigurer</code>                          | <code>@Aspect</code> + <code>pointcut</code> expressions      |
| Scope                               | All requests (even non-Spring resources like static files)           | Only requests handled by DispatcherServlet             | Any bean method, not just web requests                        |
| Typical use                         | Encoding, compression, CORS, security (Spring Security uses filters) | Auth, logging, model enrichment specific to MVC        | Transaction management, logging, caching across service layer |

#### Rule of thumb:

- Use **Filters** for concerns that must apply to *raw* requests, before Spring even gets involved (e.g., encoding, CORS, Spring Security's filter chain).
- Use **Interceptors** for concerns that need Spring MVC context (handler info, model, view).
- Use **AOP** for cross-cutting concerns on service/business-layer methods, unrelated to HTTP.

## 8. `HandlerInterceptorAdapter` (Legacy Note)



In older Spring versions (< 5.0), you extended the abstract class `HandlerInterceptorAdapter` instead of implementing `HandlerInterceptor` directly, since the interface required all three methods to be implemented.

Since Spring 5, `HandlerInterceptor` methods are `default` methods, so you can implement the interface directly and override only what you need. `HandlerInterceptorAdapter` is now **deprecated** — avoid it in new code.

---

## 9. Exception Handling Consideration

---

- If the controller throws an exception:
  - `postHandle()` is **skipped**.
  - `afterCompletion()` is **still called**, with the exception passed as the last parameter — ideal place for cleanup or logging the failure.
- Interceptors do **not** replace `@ExceptionHandler` / `@ControllerAdvice` for producing error responses; they're complementary. Use `afterCompletion` for observing/logging exceptions, not for generating the client-facing error response (that's already been committed by the time `afterCompletion` runs, in most cases).

---

## 10. Async Request Handling

---

For controllers using async processing (`Callable`, `DeferredResult`, `WebAsyncTask`), Spring provides `AsyncHandlerInterceptor` (extends `HandlerInterceptor`) with an additional method:

```
default void afterConcurrentHandlingStarted(HttpServletRequest request,
                                             HttpServletResponse response,
                                             Object handler) throws Exception {
}
```

- Called instead of `postHandle` / `afterCompletion` when the handler starts async processing.
- Important because `preHandle` / `postHandle` / `afterCompletion` behave differently with async — `postHandle` and `afterCompletion` are deferred until async processing completes.

---

## 11. Best Practices

---

1. **Keep interceptors stateless** — don't store per-request data as instance fields (interceptor beans are singletons shared across requests). Use `request.setAttribute()` to pass data



between `preHandle` / `postHandle` / `afterCompletion`.

2. **Always return a response when returning** `false` from `preHandle` — otherwise the client gets an empty/hanging response.
3. **Use** `order()` explicitly when you have multiple interceptors with dependencies on execution sequence (e.g., auth before logging).
4. **Scope with** `addPathPatterns` / `excludePathPatterns` to avoid unnecessary processing on static resources or health-check endpoints.
5. **Avoid heavy logic in** `postHandle` — prefer `preHandle` for early rejection to save processing time.
6. **Don't use interceptors for concerns needed outside Spring MVC context** (e.g., static resource filtering) — use a Filter instead.
7. Combine with `@ControllerAdvice` for centralized exception handling rather than trying to build responses inside interceptors for business exceptions.

---

## 12. Quick Summary Table

| Method                       | When it Runs                              | Can Stop Chain?                  | Has Access To                            |
|------------------------------|-------------------------------------------|----------------------------------|------------------------------------------|
| <code>preHandle</code>       | Before controller execution               | Yes (return <code>false</code> ) | Request, Response, Handler               |
| <code>postHandle</code>      | After controller, before view render      | No                               | Request, Response, Handler, ModelAndView |
| <code>afterCompletion</code> | After view is rendered / request complete | No                               | Request, Response, Handler, Exception    |

---

## 13. Common Interview Questions (Quick Reference)

- **Q: Difference between Filter and Interceptor?** → Filter works at servlet level (pre-DispatcherServlet), Interceptor works at Spring MVC level (post-DispatcherServlet) with access to handler and model.
- **Q: Can you modify the response body in an interceptor?** → Yes, technically via the `HttpServletResponse`, though it's more common/appropriate in `postHandle` for headers/model, not raw body manipulation (that's better done via Filters with wrapped streams).

- **Q: What happens if `preHandle` returns false?** → Request processing stops; controller and later interceptors are skipped; already-passed interceptors' `afterCompletion` still runs.
- **Q: Are interceptors singleton?** → Yes, by default Spring beans (including interceptors) are singleton-scoped, so avoid storing request-specific state as instance fields.